

🍯 honeypot 蜜罐系统 · 完整开发执行计划书

从管理后台到完整蜜罐系统 —— 四阶段路线图、详细任务分解、验收标准、代码规范

📅 4 个阶段

✅ 25+ 项任务

🕒 预计 10-15 周

🎯 版本目标 v1.0

目录导航

🤖 AI Agent 执行指南

📊 项目总览

第一阶段：基础修复

1.1 权限漏洞修复

1.2 CSS 主题系统重构

1.3 注册功能管控

1.4 错误页面注册

1.5 攻击事件详情页

1.6 Nginx 集成

1.7 README 完善

第一阶段验收

第二阶段：核心功能

2.1 HTTP 蜜罐服务

2.2 蜜罐节点管理

2.3 邮件通知系统

2.4 密码找回功能

2.5 RQ 异步任务队列

🤖 致执行本计划的 AI Agent

你好，开发者。这是一份 honeypot 蜜罐系统的完整开发执行计划。请仔细阅读以下说明：

- 按顺序执行：从第一阶段开始，完成一个阶段并通过验收后，再进入下一阶段。不要跳阶段。
- 先读代码：开始任何工作之前，先把当前项目的 `app/` 目录、`templates/`、`config.py`、`requirements.txt` 通读一遍，理解现有架构。
- 保持风格一致：新增代码的风格、命名、架构模式必须与现有代码保持一致。使用 Flask 蓝图模式、SQLAlchemy 模型、Jinja2 模板。
- 测试先行：每个功能完成后必须编写对应的测试用例，运行 `pytest` 确保全部通过。
- 数据库迁移：每次修改模型后，使用 `flask db migrate -m "description"` 生成迁移脚本，不要手写迁移文件。
- 遇到不确定的地方：优先参考现有代码的实现模式；如果是设计决策，选择最简单、最符合 Flask 生态的方案。
- 每完成一个任务：运行测试 → 检查 docker compose 是否能正常启动 → 更新 README 对应部分。

📊 项目总览

项目现状

当前项目是一个蜜罐系统的管理后台，基于 Flask + MySQL + Redis 构建。它提供了攻击数据接收 API、攻击者画像、数据可视化仪表盘等功能。但项目缺少实际的蜜罐服务（SSH/HTTP/Redis 蜜罐等），同时存在若干安全漏洞、前端问题和未完功能。

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

开发目标

将项目从"管理后台"升级为完整的蜜罐系统，包含：

- 多种蜜罐服务（HTTP / SSH / Telnet / Redis）
- 蜜罐节点管理与监控
- 告警规则与通知
- 完善的管理后台与数据分析
- 安全、稳定、可一键部署

技术栈

层级	技术	说明
后端框架	Flask 2.3 + Python 3.11	现有框架，继续沿用
数据库	MySQL 8.0 + SQLAlchemy + Alembic	现有方案
缓存/队列	Redis 7.2 + RQ	RQ 待接入
前端	Jinja2 模板 + ECharts + CSS 变量	CSS 主题系统需重构
蜜罐服务	Python asyncio / Twisted	新增，用 Python 保持技术栈一致
部署	Docker Compose + Nginx + Gunicorn	Nginx 待集成
邮件	Flask-Mail	新增

四阶段路线图

Phase 1

基础修复 · 1-2周

Phase 2

核心功能 · 3-4周

Phase 3

蜜罐扩展 · 4-6周

Phase 4

高级特性 · 2-3周

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

1 第一阶段：基础修复与安全加固

阶段目标

修复所有已知的安全漏洞和基础问题，让项目能安全、稳定、美观地跑起来。本阶段不做新功能，只做修复和基础完善。

7

任务数

1-2 周

预计工期

P0

最高优先级

安全

核心关注点

任务 1.1: 修复管理员权限漏洞

P0

安全

当前所有后台路由只检查 `@login_required`，不检查 `is_admin`。任何注册用户都能访问所有管理页面。必须增加管理员权限校验。

实施步骤

- 在 `app/__init__.py` 中为 `login_manager` 配置 `login_view = 'auth.login'`
- 创建自定义装饰器 `@admin_required`（放在 `app/decorators.py` 新文件中）
- 装饰器逻辑：检查 `current_user.is_authenticated` and `current_user.is_admin`，否则返回 403 或重定向
- 为 `admin_bp` 和 `settings_bp` 的所有路由加上 `@admin_required`
- 注册时 `is_admin=False` 的逻辑已正确，保持不变
- `manage.py create-admin` 创建的管理员 `is_admin=True`，保持不变

验收标准

- 普通用户 (`is_admin=False`) 访问 `/admin/dashboard` 会被拒绝 (403 或跳转提示)
- 管理员用户 (`is_admin=True`) 可正常访问所有后台页面
- 新增测试用例：测试普通用户访问后台被拒绝、管理员正常访问
- 所有现有测试仍然通过

任务 1.2: CSS 主题系统重构

P0

前端

当前 CSS 混乱：`all.css` 是深色玻璃风，但 `base.html` 默认 `light` 主题。各模板内大量内联 `<style>` 重复代码。需要统一重构为 CSS 变量驱动的双主题系统。

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

实施步骤

- 重构 `static/css/all.css`，基于 CSS 变量定义完整的主题系统
- 定义 `:root`（浅色主题）和 `[data-theme="dark"]` 两套变量
- 变量包括：背景色、文字色、边框色、卡片背景、主色调、成功/警告/错误色等
- 将各模板中的内联 `<style>` 全部移到 `all.css` 中，用 `class` 替代
- 统一设计规范：卡片圆角 12px、间距系统 4/8/12/16/20/24/32px
- `base.html` 中的主题切换按钮改为从用户配置加载初始主题，并在切换时调用 API 保存
- 确保登录页、注册页、错误页也支持主题切换

验收标准

- 浅色和深色主题都能正常显示，无文字看不清、背景看不清的问题
- 主题切换按钮点击后立即生效，刷新页面保持选择
- 所有页面（登录/注册/仪表盘/列表/详情/设置/错误页）主题一致
- 模板中不再有大段内联 `<style>`（个别特殊样式可以保留，但不能重复）
- 移动端适配正常，没有横向滚动条

任务 1.3：注册功能管控

P0

安全

蜜罐管理后台不应该开放注册。增加注册开关配置，默认关闭，由管理员在系统设置中控制。

实施步骤

- 在 `config.py` 中增加 `REGISTER_ENABLED` 配置项（默认 `False`）
- 在 `auth.py register` 路由中检查配置，关闭时返回 404 或提示“注册未开放”
- 登录页的注册链接在关闭时隐藏
- 管理员仍可通过 `manage.py create-admin` 创建账号
- 后续阶段会增加“管理员在后台创建用户”功能，本阶段先做开关

验收标准

- `REGISTER_ENABLED=False` 时，访问 `/register` 被拒绝
- `REGISTER_ENABLED=True` 时，注册功能正常工作
- 登录页在注册关闭时不显示注册链接
- 新增对应测试用例

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

任务 1.4: 错误页面注册

P1

后端

templates/errors/ 下有 404.html 和 500.html，但没有在 Flask 应用中注册 errorhandler。

实施步骤

- 在 `app/__init__.py create_app()` 函数末尾注册错误处理器
- 404 错误：返回 404 模板，状态码 404
- 500 错误：返回 500 模板，状态码 500（注意：500 页面不要用需要登录的布局）
- 美化 404/500 页面，使其与整体风格一致

验收标准

- 访问不存在的 URL 显示自定义 404 页面
- 触发服务器错误时显示自定义 500 页面
- 错误页面支持深浅主题

任务 1.5: 攻击事件详情页

P1

前端

目前只有攻击事件列表，没有单条事件的详情页。用户无法查看完整的请求 headers 和 payload。

实施步骤

- 在 `admin.py` 中新增 `/admin/attacks/<int:event_id>` 路由
- 创建 `templates/attack_detail.html` 模板
- 详情页展示：时间、IP、方法、路径、服务类型、攻击签名、严重级别、关联攻击者
- 以 JSON 格式化展示完整 headers 和 payload（支持折叠/展开）
- 攻击事件列表中，每行的 ID 或 IP 可以点击跳转到详情页

验收标准

- 访问 `/admin/attacks/1` 能看到完整事件详情
- headers 和 payload 以可读的方式展示（JSON 格式化）
- 从事件列表可以点击进入详情页
- 详情页有返回列表的按钮

任务 1.6: Nginx 集成到 Docker Compose

P1

运维

项目根目录有 `nginx.conf`，但 `docker-compose.yml` 里没有 `nginx` 服务，生产部署需要反向代理

🤖 AI Agent 执行指南

📁 项目总览

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

实施步骤

- 在 `docker-compose.yml` 中新增 nginx 服务
- 使用官方 `nginx:alpine` 镜像
- 挂载项目的 `nginx.conf` 到 `/etc/nginx/conf.d/default.conf`
- nginx 监听 80 端口，转发请求到 `app:5000`
- 设置正确的 `X-Forwarded-For` / `X-Real-IP` 头
- 静态资源由 Nginx 直接提供
- 验证：docker compose up -d 后，访问 80 端口能正常打开系统

验收标准

- docker compose up -d 一键启动所有服务（mysql/redis/app/nginx）
- 访问 `http://localhost` 能正常打开登录页（通过 Nginx 代理）
- API 接口能正确获取到客户端真实 IP（通过 `X-Forwarded-For`）
- 静态资源加载正常

任务 1.7: 完善 README 文档

P1

文档

当前 README 只有两行字。需要完整的项目介绍、架构说明、部署指南、API 文档。

实施步骤

- 项目介绍：什么是蜜罐、本项目功能概览、技术栈
- 架构图：文字描述或 Mermaid 图展示系统架构
- 快速开始：Docker Compose 一键部署步骤
- 配置说明：所有环境变量的含义和默认值
- API 文档：/api/capture 接口的调用方式、签名算法、请求示例
- 开发指南：本地开发环境搭建、运行测试、数据库迁移
- 目录结构说明

验收标准

- README.md 内容完整，结构清晰
- 新开发者能根据 README 完成本地环境搭建
- 包含 API 调用示例（curl 命令）

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

✅ 第一阶段验收清单

- ☐ 普通用户无法访问后台，管理员可以
- ☐ 浅色和深色主题都正常工作
- ☐ 注册开关功能正常，默认关闭
- ☐ 404/500 错误页面正常显示
- ☐ 攻击事件详情页可用
- ☐ docker compose 包含 Nginx 并正常代理
- ☐ README 文档完整
- ☐ pytest 全部通过，新增测试覆盖新功能
- ☐ docker compose up -d 一键启动无错误

2 第二阶段：核心功能补齐

🎯 阶段目标

让蜜罐系统有“蜜罐”——开发第一个蜜罐服务，补齐节点管理、邮件通知、异步任务等核心运营能力。完成本阶段后，系统可以作为一个最小可用的蜜罐产品运行。

7

任务数

3-4 周

预计工期

HTTP

首个蜜罐

MVP

阶段产出

任务 2.1: HTTP 蜜罐服务

P0

后端

开发第一个蜜罐服务——HTTP 诱饵蜜罐。模拟常见的 Web 管理界面（phpMyAdmin、WordPress 登录、路由器后台等），捕获攻击者的登录尝试、SQL 注入尝试、XSS 尝试等，并自动上报到管理后台。

实施步骤

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

- 在项目根目录新建 `honeypots/` 目录，作为所有蜜罐服务的代码目录
- 创建 `honeypots/http_honeypot.py`，使用 Python `http.server` 或 `Flask` 实现
- 模拟多个诱饵页面模板（放在 `honeypots/templates/`）：
 - `phpMyAdmin` 登录页
 - `WordPress /wp-admin` 登录页
 - 通用管理员登录页
- 所有页面的登录表单提交都捕获用户名/密码/IP/UA
- 自动检测常见攻击特征：SQL 注入关键字、XSS payload、路径遍历等
- 捕获到的攻击通过 `/api/capture` 上报到管理后台
- 配置项：监听端口、使用的模板、管理后台地址、API 密钥
- 在 `docker-compose.yml` 中增加 `http-honeypot` 服务
- 支持多端口多模板部署（如 8080=`phpMyAdmin`, 8081=`WordPress`）

上报数据格式

```
{
  "service": "http",
  "honeypot_type": "phpmyadmin", // 诱饵类型
  "signature": "login_attempt", // 或 sql_injection / xss_attempt
  "severity": "medium",
  "payload": {
    "username": "admin",
    "password": "123456",
    "request_path": "/phpmyadmin/index.php",
    "post_data": { ... }
  }
}
```

验收标准

- HTTP 蜜罐能独立启动并监听指定端口
- 访问诱饵页面能看到逼真的登录界面
- 提交登录表单后，管理后台能收到攻击事件
- SQL 注入尝试能被识别并标记相应的 `signature` 和 `severity`
- Docker Compose 中可一键启动 HTTP 蜜罐服务
- HTTP 蜜罐断网或管理后台不可用时，不会崩溃，有重试机制

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

任务 2.2：蜜罐节点管理

P1

后端

建立蜜罐节点的概念。支持节点注册、心跳上报、状态监控、配置管理。

实施步骤

- 新建 `HoneypotNode` 模型 (`id`, `name`, `node_key`, `ip_address`, `service_type`, `status`, `last_heartbeat`, `created_at`)
- 新增 API:
 - `/api/node/register` — 节点注册 (返回 `node_key`)
 - `/api/node/heartbeat` — 心跳上报 (更新 `last_heartbeat`)
- 管理后台新增“节点管理”页面：节点列表、状态（在线/离线）、最后心跳时间、服务类型
- 节点超过 5 分钟无心跳标记为离线
- HTTP 蜜罐启动时自动注册，运行中定期发送心跳
- 攻击事件关联到具体节点（在 `attack_events` 中增加 `node_id` 字段）

验收标准

- HTTP 蜜罐启动后自动注册，后台能看到在线节点
- 停止蜜罐服务后，后台节点状态变为离线
- 节点管理页面展示节点列表、状态、服务类型、最后心跳
- 攻击事件可以追溯到哪个节点捕获的
- 生成数据库迁移文件

任务 2.3：邮件通知系统

P1

后端

接入 Flask-Mail，实现告警邮件发送。当前 `settings` 页面的通知开关和 `tasks.py` 的日报都是空壳。

实施步骤

- 在 `requirements.txt` 中增加 `Flask-Mail`
- 在 `app/extensions.py` 中初始化 `mail`
- 在 `config.py` 中增加邮件配置项（SMTP 服务器、端口、用户名、密码、发件人、是否启用）
- 创建 `app/email_utils.py` 封装邮件发送函数
- 实现每日报告邮件：统计昨日攻击事件数、新攻击者数、Top 攻击来源，发送给开启通知的管理员
- `tasks.py generate_daily_report()` 改为实际发送邮件
- 实现告警邮件模板（HTML 格式）
- 系统设置页面增加 SMTP 配置项（后续系统设置页完成后接入，本阶段先支持环境变量配置）

第一阶段：基础修复

1.1 权限漏洞修复

1.2 CSS 主题系统重构

1.3 注册功能管控

1.4 错误页面注册

1.5 攻击事件详情页

1.6 Nginx 集成

1.7 README 完善

第一阶段验收

第二阶段：核心功能

2.1 HTTP 蜜罐服务

2.2 蜜罐节点管理

2.3 邮件通知系统

2.4 密码找回功能

2.5 RQ 异步任务队列

2.6 系统设置页面

2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

3.1 SSH 蜜罐

3.2 Telnet 蜜罐

3.3 Redis 蜜罐

验收标准

- 配置正确的 SMTP 信息后，能成功发送测试邮件
- 每日报告邮件格式美观，包含核心统计数据
- 邮件发送失败不影响主流程（try-except 包裹，记录日志）
- 新增邮件发送的测试（使用 mock）

任务 2.4：密码找回功能

P1

后端

当前登录页的"忘记密码"只是 alert。实现完整的密码找回流程：邮箱验证 + 重置链接。

实施步骤

- 新增"忘记密码"页面：输入邮箱，发送重置链接
- 新增"重置密码"页面：输入新密码，提交更新
- 使用 itsdangerous 生成带时效的 Token（默认 1 小时有效）
- 在 auth.py 中新增 forgot_password 和 reset_password 路由
- 使用上一任务实现的邮件系统发送重置链接
- 登录页的"忘记密码"按钮跳转到对应页面
- Token 验证失败或过期时给出友好提示

验收标准

- 完整的忘记密码流程可用
- 重置链接有过期时间（1 小时）
- 同一个 Token 只能使用一次
- 邮箱不存在时不泄露信息（提示"如果邮箱已注册，将收到重置邮件"）
- 新增对应测试用例

任务 2.5：RQ 异步任务队列接入

P1

后端

requirements.txt 中有 rq 但从未使用。接入 RQ 后，邮件发送、GeolIP 查询、数据清理等可以异步执行。

实施步骤

- 在 app/extensions.py 中创建 RQ queue（使用已有的 Redis 连接）
- 将邮件发送改为异步任务（入队执行，不阻塞请求）

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

- 将 GeolIP 查询改为异步（捕获攻击后先入库，再异步补全 GeolIP 信息）
- 将 `tasks.py` 中的 `cleanup_old_data` 和 `generate_daily_report` 改为 RQ job
- 在 `docker-compose.yml` 中增加 `rq-worker` 服务（运行 `worker` 进程）
- 增加简单的任务状态监控（可选，在系统设置或仪表盘显示 `worker` 状态）

验收标准

- 邮件发送不阻塞 API 响应（通过日志或时间验证）
- `docker compose` 启动后 `rq-worker` 正常运行
- 任务失败时有重试机制和错误日志
- 新增 RQ 相关测试（使用 `mock` 或 `fakeredis`）

任务 2.6：系统设置页面

P1

前端

当前 `settings` 页面只有个人设置。需要增加系统级设置页面，管理员可配置全局参数。

实施步骤

- 新建 `SystemSetting` 模型（key-value 存储方式，灵活扩展）
- 在 `settings` 页面增加“系统设置”Tab 或独立页面（仅管理员可见）
- 包含以下设置项：
 - 注册开关（`REGISTER_ENABLED`）
 - GeolIP 启用开关
 - 数据保留天数（默认 30 天）
 - API 限流速率
 - SMTP 邮件服务器配置
 - 每日报告发送时间
- 配置值缓存到 Redis 或应用内存，避免每次查库
- 修改配置后立即生效（无需重启）

验收标准

- 系统设置页面正常展示所有配置项
- 修改配置后保存成功，刷新页面显示新值
- 普通用户看不到系统设置（或无法访问）
- 注册开关等配置能实际影响对应功能
- 生成数据库迁移文件

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

任务 2.7: IP 黑名单功能

P1

后端

支持将恶意 IP 加入黑名单，标记状态，并可导出黑名单列表用于防火墙配置。

实施步骤

- 在 `AttackerProfile` 模型中增加 `is_blocked` 字段 (Boolean, 默认 False)
- 攻击者画像列表增加"封禁/解封"操作按钮
- 攻击者详情页显示封禁状态和操作按钮
- 增加"黑名单"页面：展示所有被封禁的 IP，支持批量操作
- 支持导出黑名单为 TXT / CSV (每行一个 IP，可直接用于 iptables 等)
- 被封禁 IP 的攻击事件在列表中高亮显示
- 增加封禁原因备注字段 (可选)

验收标准

- 可以在攻击者列表和详情页封禁/解封 IP
- 黑名单页面展示所有封禁 IP
- 黑名单可以导出为文件下载
- 被封禁 IP 有明显的视觉标记
- 生成数据库迁移文件

✅ 第二阶段验收清单

- ☐ HTTP 蜜罐服务可用，能捕获攻击并上报
- ☐ 蜜罐节点管理页面正常，能看到节点在线状态
- ☐ 邮件通知系统工作，每日报告邮件正常发送
- ☐ 密码找回功能完整可用
- ☐ RQ 异步任务队列正常运行
- ☐ 系统设置页面完整，配置项可修改并生效
- ☐ IP 黑名单功能完整，支持封禁/解封/导出
- ☐ pytest 全部通过
- ☐ docker compose up -d 启动所有服务无错误

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

3 第三阶段：蜜罐扩展与分析增强

🎯 阶段目标

丰富蜜罐类型（SSH / Telnet / Redis），增加告警规则引擎和操作审计，让系统从“能用”升级为“好用”。

7

任务数

4-6 周

预计工期

3 种

新增蜜罐

v0.9

阶段产出

任务 3.1: SSH 蜜罐

P1

后端

使用 Paramiko 实现 SSH 蜜罐，模拟常见的 SSH 服务，捕获暴力破解尝试和登录后的命令执行。

实施步骤

- 创建 `honeypots/ssh_honeypot.py`
- 使用 Paramiko 库实现 SSH 服务器
- 模拟 Linux SSH 登录界面（banner、提示符等）
- 捕获所有登录尝试的用户名和密码
- 登录成功后（可以配置任意密码都“成功”以诱导攻击者），提供 fake shell
- 记录攻击者执行的所有命令和输出
- 通过 API 上报攻击事件到管理后台
- 支持配置：监听端口、主机密钥、是否允许“假登录成功”
- 集成到节点管理和 docker-compose

验收标准

- SSH 蜜罐能正常监听端口，SSH 客户端可以连接
- 所有登录尝试都被捕获并上报
- 登录后执行的命令被记录

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

- 管理后台能看到 SSH 类型的攻击事件
- 蜜罐不会被攻击者利用作为跳板（限制命令执行范围，假 shell）
- docker compose 中可一键启动

任务 3.2: Telnet 蜜罐

P2

后端

模拟 Telnet 服务，捕获暴力破解。实现方式与 SSH 蜜罐类似，但使用 Telnet 协议。

实施步骤

- 创建 `honeypots/telnet_honeypot.py`
- 使用 Python telnetlib 或直接 socket 实现 Telnet 服务器
- 模拟路由器/交换机/老式 Unix 的登录界面
- 捕获用户名/密码/命令
- 通过 API 上报
- 集成到 docker-compose

验收标准

- Telnet 客户端可以连接并看到登录提示
- 登录尝试被捕获并上报
- 管理后台能看到 Telnet 类型事件

任务 3.3: Redis 蜜罐

P2

后端

模拟未授权访问的 Redis 服务，捕获攻击者的探测和攻击（如写入 SSH 公钥、主从复制 RCE 等）。

实施步骤

- 创建 `honeypots/redis_honeypot.py`
- 模拟 Redis RESP 协议，实现常见命令的假响应
- 捕获 INFO、CONFIG SET、SET 等操作
- 记录所有执行的命令和参数
- 通过 API 上报，signature 标记为 `redis_unauthorized` / `redis_exploit` 等
- 集成到 docker-compose

验收标准

第一阶段：基础修复

1.1 权限漏洞修复

1.2 CSS 主题系统重构

1.3 注册功能管控

1.4 错误页面注册

1.5 攻击事件详情页

1.6 Nginx 集成

1.7 README 完善

第一阶段验收

第二阶段：核心功能

2.1 HTTP 蜜罐服务

2.2 蜜罐节点管理

2.3 邮件通知系统

2.4 密码找回功能

2.5 RQ 异步任务队列

2.6 系统设置页面

2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

3.1 SSH 蜜罐

3.2 Telnet 蜜罐

3.3 Redis 蜜罐

- redis-cli 可以连接并执行命令
- 所有命令被捕获并上报
- 常见 Redis 攻击手段（写公钥、主从复制）被识别为高危
- 蜜罐不会泄露真实数据，不会被实际利用

任务 3.4：告警规则引擎

P1

后端

可配置的告警规则系统，满足条件时自动发送告警邮件。

实施步骤

- 新建 `AlertRule` 模型 (`id, name, rule_type, threshold, severity, enabled, created_at`)
- 支持以下规则类型：
 - **IP 频率告警**：某 IP 在 N 分钟内攻击超过 M 次
 - **高危攻击告警**：检测到指定严重级别或签名的攻击
 - **新攻击者告警**：出现从未见过的 IP
 - **新国家告警**：来自新国家/地区的攻击
- 管理后台新增“告警规则”页面：规则列表、启用/禁用、新建/编辑/删除
- 告警检测逻辑在每次攻击事件入库后执行（异步，不阻塞主流程）
- 触发告警时发送邮件通知（给开启通知的管理员）
- 记录告警历史（`AlertHistory` 模型），可在后台查看
- 避免重复告警：同一规则同一 IP 有冷却时间

验收标准

- 可以创建、编辑、删除、启用/禁用告警规则
- 满足条件时触发告警并发送邮件
- 告警历史页面可以查看所有触发过的告警
- 同一规则不会短时间内重复告警（冷却机制）
- 生成数据库迁移文件

任务 3.5：操作审计日志

P2

安全

记录所有管理员操作，支持审计追溯。

实施步骤

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善
- 第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能
- 第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

- 新建 AuditLog 模型 (id, user_id, action, detail, ip_address, user_agent, created_at)
- 记录以下操作：
 - 用户登录/登出/登录失败
 - 修改密码/修改设置
 - 封禁/解封 IP
 - 创建/修改/删除告警规则
 - 数据导出
 - 删除账号
 - 系统配置变更
- 管理后台新增"审计日志"页面：按时间倒序展示，支持筛选（操作类型、用户、时间范围）
- 审计日志不可删除（软删除或只读）

验收标准

- 关键操作都有审计记录
- 审计日志页面可以查询和筛选
- 审计日志不可被普通操作删除
- 生成数据库迁移文件

任务 3.6：数据导出增强

P2 后端

目前只有简单的 TXT 统计导出。增加多种格式和内容的导出。

实施步骤

- 攻击事件导出：支持 CSV / JSON / Excel 格式
- 支持导出当前筛选条件下的结果
- 攻击者画像导出
- 黑名单导出 (TXT/CSV)
- 导出操作记入审计日志
- 大数据量导出走异步任务 (RQ)，完成后邮件通知下载

验收标准

- 攻击事件可以导出为 CSV / JSON / Excel
- 导出内容与当前筛选条件一致
- 黑名单可以导出
- 大数量导出不阻塞 (异步)

🤖 AI Agent 执行指南

📁 项目总览

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

任务 3.7：筛选功能增强

P2

前端

丰富攻击事件和攻击者画像的筛选维度。

实施步骤

- 攻击事件列表增加筛选：
 - 时间范围（开始日期 - 结束日期）
 - 严重级别（low/medium/high）
 - 蜜罐服务类型（http/ssh/telnet/redis）
 - 攻击签名（模糊搜索）
 - 国家/地区
- 攻击者画像列表增加筛选：
 - 国家/地区
 - 是否被封禁
 - 攻击次数范围
 - 首次/最近出现时间范围
- 筛选条件在分页时保持
- 优化筛选表单的 UI 布局

验收标准

- 所有新增筛选项都能正常工作
- 多条件组合筛选结果正确
- 分页时筛选条件保持
- 移动端布局正常

✅ 第三阶段验收清单

- ☐ SSH 蜜罐可用，能捕获登录和命令
- ☐ Telnet 蜜罐可用
- ☐ Redis 蜜罐可用
- ☐ 告警规则引擎工作，能触发邮件告警
- ☐ 操作审计日志完整记录关键操作

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善
- 第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

- ❑ 数据导出支持 CSV/JSON/Excel 多格式
- ❑ 筛选功能增强，多维度组合筛选正常
- ❑ pytest 全部通过
- ❑ docker compose 一键启动所有蜜罐服务
- ❑ README 更新了新增蜜罐和功能的说明

4 第四阶段：高级特性与体验优化

🎯 阶段目标

提升产品完整度和用户体验，增加实时推送、权限管理、威胁情报等高级特性。完成本阶段后，系统达到 v1.0 正式版本标准。

6

任务数

2-3 周

预计工期

v1.0

目标版本

体验

核心关注点

任务 4.1: 实时数据推送

P2

前端

使用 WebSocket 实现新攻击事件的实时推送，仪表盘可以“动起来”。

实施步骤

- 引入 Flask-SocketIO
- 后端：攻击事件入库后，通过 SocketIO 广播新事件
- 前端仪表盘：接收实时事件，动态更新统计数字和最近事件列表
- 事件列表页：实时追加新事件（可选，带提示气泡）
- 连接断开重连机制
- 只有登录用户可以连接 WebSocket

验收标准

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

- 打开仪表盘后，新攻击发生时数字会实时增加
- 最近攻击事件列表会实时追加新记录
- 网络断开后自动重连
- 未登录用户无法建立 WebSocket 连接

任务 4.2：用户权限管理

P2 后端

完善用户系统，支持多角色权限管理。

实施步骤

- 定义角色系统：超级管理员（super_admin）、普通管理员（admin）、只读观察员（viewer）
- 权限矩阵：
 - 超级管理员：所有权限 + 用户管理 + 系统设置
 - 普通管理员：查看数据 + 封禁 IP + 告警规则管理
 - 只读观察员：只能查看数据，不能修改任何东西
- 新增"用户管理"页面：用户列表、创建用户、编辑角色、启用/禁用用户、重置密码
- 用装饰器或中间件实现权限校验
- 注册功能默认关闭，用户由管理员创建

验收标准

- 不同角色用户登录后看到的菜单和功能不同
- 低权限用户访问高权限页面被拒绝
- 超级管理员可以创建/编辑/禁用用户
- 权限修改后立即生效
- 生成数据库迁移文件

任务 4.3：威胁情报对接

P2 后端

对接威胁情报源，自动标记已知的恶意 IP，丰富攻击者画像。

实施步骤

- 支持至少一个免费威胁情报源（如 abuseipdb.com API 或其他开源 feeds）
- 在攻击者画像中显示威胁情报标签（恶意 IP / 已知扫描器 / 代理节点等）
- 定期批量查询或实时查询新 IP

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

- 威胁情报数据缓存到本地数据库，避免重复查询
- 系统设置中可配置 API Key 和启用/禁用
- 仪表盘增加“已知威胁 IP 占比”统计

验收标准

- 已知恶意 IP 在攻击者画像中有明显标记
- 威胁情报数据本地缓存，不重复消耗 API 配额
- 查询失败不影响主流程
- 可以在系统设置中开启/关闭

任务 4.4: ECharts 本地化

P2

前端

当前所有图表都依赖 jsdelivr CDN。离线或内网环境下不可用。将 ECharts 和世界地图数据下载到本地。

实施步骤

- 下载 echarts.min.js 到 `static/js/` 目录
- 下载 world.json 地图数据到 `static/json/` 目录
- 将所有模板中的 CDN 引用替换为本地路径
- 验证所有图表在断网情况下仍能正常显示

验收标准

- 所有图表和地图在无外网环境下正常显示
- 功能和样式与使用 CDN 时一致
- 模板中不再有 jsdelivr CDN 引用

任务 4.5: 模拟攻击表单增强

P2

前端

后端 `simulate_attack` 支持自定义参数，但前端只有一个随机按钮。增加可配置的模拟攻击表单。

实施步骤

- 在仪表盘或独立页面增加模拟攻击表单
- 可配置字段：IP 地址、User-Agent、国家、严重级别、蜜罐服务类型、攻击签名
- 支持批量生成（一次生成 N 条随机数据）
- 保留一键随机按钮作为快捷方式

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

验收标准

- 可以自定义参数生成模拟攻击
- 可以批量生成多条数据
- 生成后跳转到对应列表或显示成功提示

任务 4.6：测试覆盖提升

P2

测试

补充各模块的单元测试和集成测试，提升代码质量保障。

实施步骤

- 补充管理页面测试（仪表盘、攻击列表、攻击者列表、地图、统计页）
- 补充设置模块测试（修改密码、通知设置、主题切换）
- 补充数据导出测试
- 补充模拟攻击测试
- 补充告警规则测试
- 补充用户权限测试
- 测试覆盖率目标：核心业务代码 > 70%

验收标准

- pytest 全部通过
- 核心业务逻辑测试覆盖率 > 70%
- 新增功能都有对应的测试用例

✅ 第四阶段验收清单 (v1.0 发布标准)

- ☐ 仪表盘实时数据推送正常
- ☐ 用户权限管理完整（多角色 + 用户管理页面）
- ☐ 威胁情报对接可用
- ☐ ECharts 完全本地化，离线可用
- ☐ 模拟攻击表单支持自定义参数和批量生成
- ☐ 测试覆盖率 > 70%

- ☐ 所有文档 (README / API 文档) 完整且最新
- ☐ docker compose up -d 一键启动全部服务无错误
- ☐ 安全审计通过, 无已知高危漏洞
- ☐ v1.0 版本发布 🎉

📐 代码规范与开发约定

后端 (Python / Flask)

- 遵循 **PEP 8** 代码风格
- 使用 4 空格缩进, 行宽不超过 120 字符
- 所有路由放在 `app/blueprints/` 下, 按模块拆分
- 所有模型放在 `app/models.py` (太多时可以拆分为 `models/` 目录)
- 数据库修改必须通过 `flask db migrate` 生成迁移文件, **禁止直接修改数据库**
- 敏感信息 (密钥、密码) 必须通过环境变量配置, 不能硬编码
- 使用 Python 类型注解 (type hints)
- 错误处理: 用户操作错误返回 flash 消息; API 错误返回 JSON + 合适的 HTTP 状态码

前端 (Jinja2 / CSS / JS)

- 模板继承 `base.html`, 页面内容放在 `{% block content %}` 中
- CSS 统一写在 `static/css/all.css` 中, 使用 BEM 或简洁的 class 命名
- 主题相关颜色使用 CSS 变量, 不要硬编码颜色值
- 避免在模板中写大段 `<style>`, 特殊样式尽量提炼到 class
- JavaScript 尽量少用, 复杂交互用 ECharts 或轻量脚本
- 所有表单包含 CSRF token (`{{ csrf_token() }}` 或 `{{ form.hidden_tag() }}`)

Git 提交规范

提交信息格式: `类型: 简要描述`

第一阶段：基础修复

- 1.1 权限漏洞修复
- 1.2 CSS 主题系统重构
- 1.3 注册功能管控
- 1.4 错误页面注册
- 1.5 攻击事件详情页
- 1.6 Nginx 集成
- 1.7 README 完善

第一阶段验收

第二阶段：核心功能

- 2.1 HTTP 蜜罐服务
- 2.2 蜜罐节点管理
- 2.3 邮件通知系统
- 2.4 密码找回功能
- 2.5 RQ 异步任务队列
- 2.6 系统设置页面
- 2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

- 3.1 SSH 蜜罐
- 3.2 Telnet 蜜罐
- 3.3 Redis 蜜罐

- 提交信息格式： `类型：简要描述`
- 类型： feat (新功能)、fix (修复)、refactor (重构)、style (样式)、docs (文档)、test (测试)、chore (构建/工具)
- 示例： `feat: add SSH honeypot service`、`fix: admin permission check on dashboard`

🧪 测试要求

测试框架

使用 pytest 进行测试，测试文件放在 `tests/` 目录。

运行测试

```
pytest tests/ -v
```

测试分类

测试类型	说明	示例
模型测试	测试数据模型的 CRUD、关联关系、业务方法	<code>test_user_create_and_password</code>
API 测试	测试 API 接口的请求/响应、鉴权、参数校验	<code>test_valid_signature_returns_200</code>
页面测试	测试页面能否正常渲染、登录保护、数据展示	<code>test_dashboard_requires_login</code>
集成测试	测试多步业务流程	<code>test_register_then_login</code>

测试配置

- 使用 SQLite 内存数据库进行测试 (已在 TestConfig 中配置)
- 测试中关闭 GeoIP 和限流，避免外部依赖
- 邮件发送使用 mock，不发送真实邮件
- 每个测试用例之间数据隔离 (fixture 中创建和销毁表)

2.2 蜜罐节点管理

2.3 邮件通知系统

2.4 密码找回功能

2.5 RQ 异步任务队列

2.6 系统设置页面

2.7 IP 黑名单功能

第二阶段验收

第三阶段：蜜罐扩展

3.1 SSH 蜜罐

3.2 Telnet 蜜罐

3.3 Redis 蜜罐

3.4 告警规则引擎

3.5 操作审计日志

3.6 数据导出增强

3.7 筛选功能增强

第三阶段验收

第四阶段：高级特性

4.1 实时数据推送

4.2 用户权限管理

4.3 威胁情报对接

4.4 ECharts 本地化

4.5 模拟攻击表单增强

4.6 测试覆盖提升

第四阶段验收

代码规范

测试要求

最终交付清单 (v1.0)

功能交付

- ✓ 管理后台 (仪表盘 / 攻击事件 / 攻击者画像 / 统计 / 地图 / 设置)
- ✓ 4 种蜜罐服务 (HTTP / SSH / Telnet / Redis)
- ✓ 蜜罐节点管理与监控
- ✓ 告警规则引擎与邮件通知
- ✓ IP 黑名单管理
- ✓ 用户权限管理 (多角色)
- ✓ 操作审计日志
- ✓ 实时数据推送
- ✓ 威胁情报对接
- ✓ 数据多格式导出

代码交付

- 完整的项目源代码
- 数据库迁移文件
- 测试用例 (核心覆盖率 > 70%)
- Docker Compose 一键部署配置

文档交付

- README.md (项目介绍 + 快速开始 + 配置说明)
- API 文档 (数据上报接口 + 节点管理接口)
- 部署指南
- 蜜罐服务配置说明



恭喜

恭喜您顺利完成本项目交付！本项目交付物包括：完整源代码、数据库迁移文件、测试用例、部署指南、蜜罐服务配置说明、API 文档、README.md 文档等。所有交付物均已按照约定时间完成，并已通过最终验收。感谢您的信任与支持，我们将持续为您提供优质服务。

完成以上所有内容后，项目就从一个半成品的管理后台，升级为一个功能完整、安全可靠、可部署运行的企业级蜜罐系统 v1.0。

📦 最终交付清单